

Python - break Statement

Python break Statement

Python **break statement** is used to terminate the current **loop** and resumes execution at the next statement, just like the traditional **break statement in C**.

The most common use for Python break statement is when some external condition is triggered requiring a sudden exit from a loop. The **break statement** can be used in both Python **while** and **for** loops.

If you are using **nested loops in Python**, the break statement stops the execution of the innermost loop and start executing the next line of code after the block.

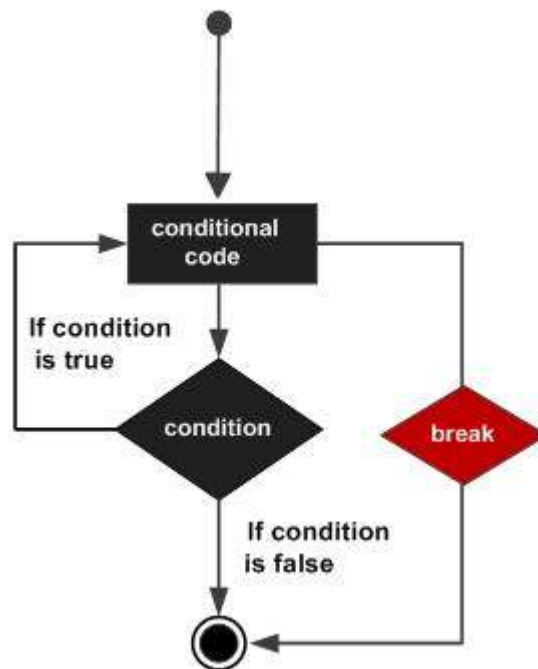
Syntax of break Statement

The syntax for a **break statement** in Python is as follows –

```
looping statement:
    condition check:
        break
```

Flow Diagram of break Statement

Following is the flowchart of the break statement –



break Statement with for loop

If we use break statement inside a **for loop**, it interrupts the normal flow of program and exit the loop before completing the iteration.

Example

In this example, we will see the working of break statement in for loop.

```

for letter in 'Python':
    if letter == 'h':
        break
    print ("Current Letter :", letter)
print ("Good bye!")
  
```

When the above code is executed, it produces the following result –

```

Current Letter : P
Current Letter : y
Current Letter : t
Good bye!
  
```

break Statement with while loop

Similar to the **for loop**, we can use the **break** statement to skip the code inside **while loop** after the specified condition becomes TRUE.

Example

The code below shows how to use **break** statement with **while** loop.

```
var = 10
while var > 0:
    print ('Current variable value :', var)
    var = var -1
    if var == 5:
        break

print ("Good bye!")
```

On executing the above code, it produces the following result –

```
Current variable value : 10
Current variable value : 9
Current variable value : 8
Current variable value : 7
Current variable value : 6
Good bye!
```

break Statement with Nested Loops

In **nested loops**, one loop is defined inside another. The loop that enclose another loop (i.e. inner loop) is called as **outer loop**.

When we use a **break** statement with nested loops, it behaves as follows –

- When **break** statement is used inside the inner loop, only the inner loop will be skipped and the program will continue executing statements after the inner loop
- And, when the **break** statement is used in the outer loop, both the outer and inner loops will be skipped and the program will continue executing statements immediate to the outer loop.

Example

The following program demonstrates the use of break in a **for loop** iterating over a **list**. Here, the specified number will be searched in the list. If it is found, then the loop terminates with the "found" message.

```
no = 33
numbers = [11,33,55,39,55,75,37,21,23,41,13]
for num in numbers:
    if num == no:
        print ('number found in list')
        break
    else:
        print ('number not found in list')
```

The above program will produce the following **output** –

```
number found in list
```

TOP TUTORIALS

[Python Tutorial](#)[Java Tutorial](#)[C++ Tutorial](#)[C Programming Tutorial](#)[Chapters](#) ▾[Categories](#) ≡[R Tutorial](#)[HTML Tutorial](#)[CSS Tutorial](#)[JavaScript Tutorial](#)[SQL Tutorial](#)

TRENDING TECHNOLOGIES

Cloud Computing Tutorial
Amazon Web Services Tutorial
Microsoft Azure Tutorial
Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler
Online PHP Compiler
Online MATLAB Compiler
Online Bash Terminal
Online SQL Compiler

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.